ETL Source Code

etl.py

```
"""
This is the orchestration script.
"""

from dataclasses import asdict
from datetime import datetime, timedelta
from pathlib import Path

from airflow.sdk import Param, dag, task, task_group

import etl_config as config
from etl import dump, extract, load, transform


@dag(
    dag_id="dutch_railway_etl",
    description="Extract the NS services yearly file, transform it into the star schema, load
 ↪  it into Postgres",
    schedule=None,
    start_date=datetime(2025, 1, 1),
    catchup=False,
    params={"year": Param(config.DEFAULT_YEAR, type="integer", minimum=2019)},
    default_args={"retries": 2, "retry_delay": timedelta(minutes=2)},
    tags=["dutch_railway", "etl"],
)
def dutch_railway_etl():
    @task_group(group_id="extract")
    def extract_group():
        @task
        def fetch_remote_metadata(params=None) -> dict:
            remote = extract.fetch_remote_metadata(config.source_url(params["year"]))
            return asdict(remote)

        @task
        def download(remote: dict, params=None) -> str:
            path = extract.download(extract.RemoteFile(**remote),
 ↪  config.raw_path(params["year"]))
            return str(path)

        @task
        def validate_header(raw: str) -> str:
            extract.validate_header(Path(raw))
            return raw
```

```
        @task
        def stage_to_parquet(raw: str, params=None) -> str:
            return str(extract.stage_to_parquet(Path(raw), config.staged_path(params["year"])))

        @task
        def profile_staged(staged: str, params=None) -> dict:
            return {"staged": staged, **extract.profile_staged(Path(staged), params["year"])}

        raw = validate_header(download(fetch_remote_metadata()))
        return profile_staged(stage_to_parquet(raw))

    # [ to: @zrygan, fr: @all
    #   Currently this is just a @task
    #   if needed bump this up into a @task_group
    # ]
    @task
    def transform_step(extracted: dict, params=None) -> dict:
        return transform.run_transform(Path(extracted["staged"]),
         ↪  config.warehouse_dir(params["year"]))

    # [ to: @zrygan, fr: @all
    #   Currently this is just a @task
    #   if needed bump this up into a @task_group
    # ]
    @task
    def load_step(transformed: dict, params=None) -> dict:
        return load.load_warehouse(Path(transformed["warehouse_dir"]), params["year"])

    @task
    def dump_step(loaded: dict) -> dict:
        return dump.dump_database()

    dump_step(load_step(transform_step(extract_group())))


dutch_railway_etl()
```

config/etl_config.py

```
"""Configuration settings and path helpers for the Dutch Railway ETL pipeline."""

import os
from pathlib import Path

CONTAINER_DATA_DIR = Path("/opt/airflow/data")  # where compose mounts dutch_railway/data
DATA_DIR = Path(os.environ.get("DUTCH_RAILWAY_DATA_DIR", CONTAINER_DATA_DIR))
RAW_DIR = DATA_DIR / "raw"
STAGING_DIR = DATA_DIR / "staging"
WAREHOUSE_DIR = DATA_DIR / "warehouse"
DUMP_DIR = DATA_DIR / "dumps"
DUMP_PATH = DUMP_DIR / "dutch_railway_db.dump"

DB_URL = os.environ.get(
```

```
    "DUTCH_RAILWAY_DB_URL",
    "postgresql://dutch_railway_user:dutch_railway_password@localhost:5433/dutch_railway_db",
)

SOURCE_URL_TEMPLATE = "https://blobs.duckdb.org/nl-railway/services-{year}.csv.gz"
DEFAULT_YEAR = 2025

SOURCE_TIMEZONE = "Europe/Amsterdam"
DUCKDB_MEMORY_LIMIT = os.environ.get("DUTCH_RAILWAY_DUCKDB_MEMORY_LIMIT", "2GB")
DUCKDB_THREADS = int(os.environ.get("DUTCH_RAILWAY_DUCKDB_THREADS", "4"))

SERVICES_COLUMNS: dict[str, str] = {
    "Service:RDT-ID": "BIGINT",
    "Service:Date": "DATE",
    "Service:Type": "VARCHAR",
    "Service:Company": "VARCHAR",
    "Service:Train number": "BIGINT",
    "Service:Completely cancelled": "BOOLEAN",
    "Service:Partly cancelled": "BOOLEAN",
    "Service:Maximum delay": "BIGINT",
    "Stop:RDT-ID": "BIGINT",
    "Stop:Station code": "VARCHAR",
    "Stop:Station name": "VARCHAR",
    "Stop:Arrival time": "TIMESTAMPTZ",
    "Stop:Arrival delay": "BIGINT",
    "Stop:Arrival cancelled": "BOOLEAN",
    "Stop:Departure time": "TIMESTAMPTZ",
    "Stop:Departure delay": "BIGINT",
    "Stop:Departure cancelled": "BOOLEAN",
    "Stop:Platform change": "BOOLEAN",
    "Stop:Planned platform": "VARCHAR",
    "Stop:Actual platform": "VARCHAR",
}

TABLES = ["dim_date", "dim_time", "dim_station", "fact_service", "fact_stop"]

STATION_CODE_MERGES: dict[str, str] = {"BERHBT": "BERHBL"}
STATION_CODE_RENAMES: dict[tuple[str, str], str] = {("LAND", "Venezia Santa Lucia"): "VENEZ"}
TIME_OF_DAY: dict[str, tuple[int, int]] = {"morning": (5, 12), "afternoon": (12, 18), "night":
 ↪  (18, 5)}


def source_url(year: int) -> str:
    """Format remote download URL for a given dataset year.

    Args:
        year: Calendar year of the NS services dataset.

    Returns:
        str: Fully-qualified HTTP download URL.
    """
    return SOURCE_URL_TEMPLATE.format(year=year)


def raw_path(year: int) -> Path:
    """Return local filesystem path for the raw compressed services archive.
```

```
    Args:
        year: Calendar year of the NS services dataset.

    Returns:
        Path: Path to the destination `.csv.gz` file.
    """
    return RAW_DIR / f"services-{year}.csv.gz"


def staged_path(year: int) -> Path:
    """Return local filesystem path for the staged typed Parquet file.

    Args:
        year: Calendar year of the NS services dataset.

    Returns:
        Path: Path to the destination staged `.parquet` file.
    """
    return STAGING_DIR / f"services-{year}.parquet"


def warehouse_dir(year: int) -> Path:
    """Return directory path for transformed warehouse Parquet tables.

    Args:
        year: Calendar year of the NS services dataset.

    Returns:
        Path: Directory path containing the dimensional star schema tables.
    """
    return WAREHOUSE_DIR / str(year)
```

etl/extract.py

```
import gzip
import json
import logging
import os
import shutil
import urllib.request
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path

import duckdb

import etl_config as config

log = logging.getLogger(__name__)

_CHUNK = 1024 * 1024
```

```
@dataclass(frozen=True)
class RemoteFile:
    """Remote file metadata obtained before downloading.

    Attributes:
        url: Source download URL.
        size: Expected file size in bytes from Content-Length.
        etag: HTTP ETag entity tag for cache validation, if provided.
        last_modified: HTTP Last-Modified header value, if provided.
    """

    url: str
    size: int
    etag: str | None
    last_modified: str | None


def fetch_remote_metadata(url: str, timeout: int = 30) -> RemoteFile:
    """Query remote file headers to inspect size, ETag, and modification timestamp.

    Performs an HTTP HEAD request to determine file version and payload size prior
    to downloading. Enables idempotent caching, truncated transfer detection, and
    early failure if the remote resource is missing.

    Args:
        url: Source file URL to inspect.
        timeout: Network request timeout in seconds. Defaults to 30.

    Returns:
        RemoteFile: Metadata parsed from HTTP response headers.

    Raises:
        RuntimeError: If the remote server does not provide a Content-Length header.
        urllib.error.URLError: If the HTTP request fails or times out.
    """
    req = urllib.request.Request(url, method="HEAD", headers={"User-Agent": "mono4db-etl"})
    with urllib.request.urlopen(req, timeout=timeout) as resp:
        size = resp.headers.get("Content-Length")
        if size is None:
            raise RuntimeError(f"{url} did not report Content-Length; cannot verify download")
        remote = RemoteFile(
            url=url,
            size=int(size),
            etag=(resp.headers.get("ETag") or "").strip('"') or None,
            last_modified=resp.headers.get("Last-Modified"),
        )
    log.info("Remote %s: %.1f MB, etag=%s", url, remote.size / 1e6, remote.etag)
    return remote


def _meta_path(dest: Path) -> Path:
    """Return the companion metadata JSON path for a target file.

    Args:
        dest: Target destination file path.
```

```
    Returns:
        Path: Path to the associated sidecar metadata file (.meta.json).
    """
    return dest.with_name(dest.name + ".meta.json")


def download(remote: RemoteFile, dest: Path, force: bool = False, timeout: int = 60) -> Path:
    """Stream a remote file to disk with atomic writing and version validation.

    Downloads the file in 1 MiB chunks to maintain constant memory usage. Writes
    to a temporary `<dest>.part` file before atomically replacing `dest` upon
    successful byte-count verification. Skips download if a matching sidecar
    metadata file (`<dest>.meta.json`) and identical file size already exist.

    Args:
        remote: Remote file metadata containing URL, size, and ETag.
        dest: Target local file path.
        force: If True, bypass cache and force re-download. Defaults to False.
        timeout: Socket read timeout in seconds. Defaults to 60.

    Returns:
        Path: Path to the verified local file (`dest`).

    Raises:
        RuntimeError: If downloaded byte count does not match `remote.size`.
        urllib.error.URLError: If the connection drops or times out.
    """
    dest.parent.mkdir(parents=True, exist_ok=True)
    meta_path = _meta_path(dest)

    if not force and dest.exists() and meta_path.exists():
        cached = json.loads(meta_path.read_text())
        if (
            cached.get("etag") == remote.etag
            and cached.get("size") == remote.size
            and dest.stat().st_size == remote.size
        ):
            log.info("Up to date, skipping download: %s", dest)
            return dest

    part = dest.with_name(dest.name + ".part")
    req = urllib.request.Request(remote.url, headers={"User-Agent": "mono4db-etl"})
    log.info("Downloading %s -> %s", remote.url, dest)
    with urllib.request.urlopen(req, timeout=timeout) as resp, open(part, "wb") as out:
        shutil.copyfileobj(resp, out, length=_CHUNK)

    got = part.stat().st_size
    if got != remote.size:
        part.unlink(missing_ok=True)
        raise RuntimeError(f"Truncated download: got {got} bytes, expected {remote.size}")

    os.replace(part, dest)
    meta_path.write_text(
        json.dumps(
            {**asdict(remote), "downloaded_at": datetime.now(timezone.utc).isoformat()},
            indent=2,
```

```
        )
    )
    return dest


def validate_header(csv_gz: Path, expected: list[str] | None = None) -> list[str]:
    """Validate that the compressed CSV header matches expected columns in order.

    Reads only the first line of the gzip file to quickly detect schema drift
    or corrupt archives before full ingestion.

    Args:
        csv_gz: Path to the gzipped CSV file.
        expected: Ordered list of expected column names. Defaults to
            `config.SERVICES_COLUMNS` keys.

    Returns:
        list[str]: The parsed column names from the file header.

    Raises:
        ValueError: If column names, count, or ordering differ from expected.
        gzip.BadGzipFile: If the file is not a valid gzip archive.
    """
    expected = expected or list(config.SERVICES_COLUMNS)
    with gzip.open(csv_gz, "rt", encoding="utf-8", newline="") as f:
        header = f.readline().rstrip("\r\n").split(",")

    if header != expected:
        missing = [c for c in expected if c not in header]
        extra = [c for c in header if c not in expected]
        raise ValueError(
            f"Header mismatch in {csv_gz.name}: missing={missing} extra={extra} "
            f"(order differs: {not missing and not extra})"
        )
    log.info("Header OK: %d columns", len(header))
    return header


def _sql_str(value: str | Path) -> str:
    """Escape and quote a string or path literal for DuckDB SQL.

    Args:
        value: String or Path to quote.

    Returns:
        str: Single-quoted string literal with internal single quotes doubled.
    """
    return "'" + str(value).replace("'", "''") + "'"


def _connect() -> duckdb.DuckDBPyConnection:
    """Initialize an in-memory DuckDB connection configured for streaming extract.

    Sets configured time zone, memory limits, and thread counts. Disables
    `preserve_insertion_order` to optimize streaming throughput and minimize
    memory buffering.
```

```
    Returns:
        duckdb.DuckDBPyConnection: Configured DuckDB database connection.
    """
    con = duckdb.connect()
    con.execute(f"SET TimeZone = {_sql_str(config.SOURCE_TIMEZONE)}")
    con.execute(f"SET memory_limit = {_sql_str(config.DUCKDB_MEMORY_LIMIT)}")
    con.execute(f"SET threads = {config.DUCKDB_THREADS}")
    con.execute("SET preserve_insertion_order = false")
    return con


def stage_to_parquet(csv_gz: Path, parquet: Path) -> Path:
    """Convert raw gzipped CSV data into typed, compressed Parquet via DuckDB.

    Streams CSV row groups into Parquet without loading full datasets into RAM,
    using explicit schema types from `config.SERVICES_COLUMNS` to prevent type
    guessing errors. Output is written atomically via a `.part` file with zstd
    compression and pinned to `config.SOURCE_TIMEZONE`.

    Args:
        csv_gz: Path to the raw source `.csv.gz` file.
        parquet: Destination path for the staged `.parquet` file.

    Returns:
        Path: Path to the staged Parquet file.
    """
    parquet.parent.mkdir(parents=True, exist_ok=True)
    part = parquet.with_name(parquet.name + ".part")
    columns = "{" + ", ".join(f"{_sql_str(k)}: {_sql_str(v)}" for k, v in
↪   config.SERVICES_COLUMNS.items()) + "}"

    con = _connect()
    try:
        con.execute(
            f"""
            COPY (
                FROM read_csv(
                    {_sql_str(csv_gz)},
                    header = true,
                    auto_detect = false,
                    delim = ',',
                    quote = '"',
                    compression = 'gzip',
                    columns = {columns}
                )
            ) TO {_sql_str(part)} (FORMAT parquet, COMPRESSION zstd)
            """
        )
    finally:
        con.close()

    os.replace(part, parquet)
    log.info("Staged %s (%.1f MB)", parquet, parquet.stat().st_size / 1e6)
    return parquet
```

```python
def profile_staged(parquet: Path, year: int) -> dict:
    """Run data quality checks on the staged Parquet file and return metrics.

    Verifies essential invariants: ensures the dataset is non-empty, confirms all
    `Service:Date` values match the specified year, and validates that `Stop:RDT-ID`
    is strictly unique. Returns a lightweight summary dictionary suitable for
    Airflow XCom.

    Args:
        parquet: Path to the staged Parquet file.
        year: Expected calendar year of the dataset.

    Returns:
        dict: Summary statistics including row count, distinct entities,
            date bounds, and null timestamp counts.

    Raises:
        ValueError: If any validation checks fail (empty dataset, year mismatch,
            or duplicate stop IDs).
    """
    con = _connect()
    try:
        row = con.execute(
            f"""
            SELECT
                count(*)                                    AS rows,
                count(DISTINCT "Stop:RDT-ID")               AS stop_ids,
                count(DISTINCT "Service:RDT-ID")            AS services,
                count(DISTINCT "Stop:Station code")         AS stations,
                min("Service:Date")                         AS min_date,
                max("Service:Date")                         AS max_date,
                count(DISTINCT "Service:Date")              AS days,
                count(*) FILTER ("Stop:Arrival time" IS NULL)   AS null_arrival_time,
                count(*) FILTER ("Stop:Departure time" IS NULL) AS null_departure_time
            FROM read_parquet({_sql_str(parquet)})
            """
        ).fetchone()
        cols = [d[0] for d in con.description]
    finally:
        con.close()

    summary = dict(zip(cols, row))
    summary["min_date"] = str(summary["min_date"])
    summary["max_date"] = str(summary["max_date"])

    problems = []
    if summary["rows"] == 0:
        problems.append("file has no rows")
    if not (summary["min_date"].startswith(str(year)) and
     ↪  summary["max_date"].startswith(str(year))):
        problems.append(f"dates {summary['min_date']}..{summary['max_date']} outside {year}")
    if summary["stop_ids"] != summary["rows"]:
        problems.append(f"Stop:RDT-ID not unique ({summary['rows'] - summary['stop_ids']}
↪   duplicates)")
    if problems:
```

```
        raise ValueError(f"Staged file failed checks: {'; '.join(problems)}. Summary:
        ↪ {summary}")

    log.info("Profile: %s", summary)
    return summary


def run_extract(year: int = config.DEFAULT_YEAR, force: bool = False) -> dict:
    """Execute the complete extract pipeline sequentially in a single process.

    Intended for standalone CLI and interactive notebook testing. Airflow
    orchestrates these stages as individual tasks in production.

    Args:
        year: Dataset year to extract. Defaults to `config.DEFAULT_YEAR`.
        force: If True, forces re-downloading even if cached locally.
            Defaults to False.

    Returns:
        dict: File paths for raw and staged datasets along with profiling summary.
    """
    remote = fetch_remote_metadata(config.source_url(year))
    raw = download(remote, config.raw_path(year), force=force)
    validate_header(raw)
    staged = stage_to_parquet(raw, config.staged_path(year))
    return {"raw": str(raw), "staged": str(staged), **profile_staged(staged, year)}
```

etl/transform.py

```
"""Transform stage for the Dutch Railway ETL pipeline.

Transforms staged services Parquet data into a star schema comprising
dimensional tables (dim_date, dim_time, dim_station) and fact tables
(fact_service, fact_stop).

Schema questions raised against the EDA notebook, and the answers agreed on before
↪ implementing:

Q1. FACT_STOP has one `next_day` flag, but stop days relative to Service:Date are 0 (~17M),
    +1 (~818k), -1 (13), +2/+3/+6 (a handful) and +92/+107 (11 stops, likely bad data), and
    11,503 stops arrive on day 0 but depart on day +1. How should this be stored?
A1. Two int offsets: arrival_day_offset and departure_day_offset (days after service_date).
    No next_day flag.

Q2. Stop cancel_status (completely / departure-only / arrival-only / none): the origin has no
    arrival and the terminus no departure (NULL). How are a cancelled origin (116k stops) and a
    cancelled terminus (119k stops) classified?
A2. As 'completely': a missing side counts as cancelled when the other side is, the same as
    the EDA's "stop cancelled" rule, coalesce(a, true) AND coalesce(d, true).

Q3. Cancelled stops always have delay 0, and some services never report a delay. Store those
    delays as NULL?
A3. NULL for cancelled arrivals/departures only. Everything else is kept as is.
```

```
Q4. Service:Company changes format on 14 Dec (DB -> 'Deutsche B', numeric codes, suffixed
    names). Normalize it?
A4. Keep it raw. A mapping can be a later step.
"""
import logging
import os
from pathlib import Path

import etl_config as config
from etl.extract import _connect, _sql_str

log = logging.getLogger(__name__)


def _station_code_sql() -> str:
    """Generate SQL CASE expression applying station code renames and merges.

    Returns:
        str: DuckDB SQL CASE expression returning the canonical station code.
    """
    whens = [
        f'WHEN "Stop:Station code" = {_sql_str(code)} AND "Stop:Station name" =
         ↪  {_sql_str(name)} THEN {_sql_str(new)}'
        for (code, name), new in config.STATION_CODE_RENAMES.items()
    ] + [
        f'WHEN "Stop:Station code" = {_sql_str(old)} THEN {_sql_str(new)}'
        for old, new in config.STATION_CODE_MERGES.items()
    ]
    return f'CASE {" ".join(whens)} ELSE "Stop:Station code" END'


def _time_of_day_sql(hour: str) -> str:
    """Generate SQL CASE expression categorizing an hour into a time-of-day bucket.

    Args:
        hour: SQL expression or column name representing the hour integer (0-23).

    Returns:
        str: DuckDB SQL CASE expression returning the time-of-day bucket label.
    """
    whens = []
    for label, (start, end) in config.TIME_OF_DAY.items():
        cond = f"{hour} >= {start} AND {hour} < {end}" if start < end else f"({hour} >=
 ↪  {start} OR {hour} < {end})"
        whens.append(f"WHEN {cond} THEN {_sql_str(label)}")
    return f"CASE {' '.join(whens)} END"


def _time_key(ts: str) -> str:
    """Generate SQL expression converting a timestamp column into an HHMM integer key.

    Args:
        ts: SQL expression or column name representing a timestamp.

    Returns:
        str: DuckDB SQL expression yielding an HHMM SMALLINT key, or NULL if ts is NULL.
```

```
    """
    return f"(hour({ts}) * 100 + minute({ts}))::SMALLINT"


def _date_key(d: str) -> str:
    """Generate SQL expression converting a date column into a YYYYMMDD integer key.

    Args:
        d: SQL expression or column name representing a date.

    Returns:
        str: DuckDB SQL expression yielding a YYYYMMDD INTEGER key.
    """
    return f"strftime({d}, '%Y%m%d')::INTEGER"


_QUERIES: dict[str, str] = {
    "dim_date": f"""
        SELECT {_date_key("d")}  AS date_key,
               d::DATE           AS service_date,
               week(d)::SMALLINT AS week,     -- ISO week: 29-31 Dec 2025 are week 1
               month(d)::SMALLINT AS month,
               quarter(d)::SMALLINT AS quarter
        FROM range(
            (SELECT min("Service:Date") FROM stops),
            (SELECT max("Service:Date") FROM stops) + INTERVAL 1 DAY,
            INTERVAL 1 DAY
        ) t(d)
    """,
    "dim_time": f"""
        SELECT (h * 100 + m)::SMALLINT AS time_key,
               h::SMALLINT             AS hour,
               m::SMALLINT             AS minute,
               {_time_of_day_sql("h")} AS time_of_day
        FROM range(24) hs(h), range(60) ms(m)
    """,
    "dim_station": """
        SELECT station_code, any_value(station_name) AS station_name
        FROM stops GROUP BY station_code
    """,

    "fact_service": f"""
        SELECT "Service:RDT-ID"                          AS rdt_id,
               {_date_key('any_value("Service:Date")')} AS date_key,
               arg_min("Service:Train number", stop_order) AS first_train_number,
               arg_max("Service:Train number", stop_order) AS last_train_number,
               count(DISTINCT "Service:Train number") > 1  AS train_id_changed,
               CASE WHEN any_value("Service:Completely cancelled") THEN 'completely'
                    WHEN any_value("Service:Partly cancelled")     THEN 'partially'
                    ELSE 'none' END                     AS cancel_status,
               any_value("Service:Type")                AS service_type,
               any_value("Service:Company")             AS company_name
        FROM stops
        GROUP BY "Service:RDT-ID"
    """,
```

```
    "fact_stop": f"""
        SELECT "Stop:RDT-ID"                            AS stop_id,
               "Service:RDT-ID"                         AS rdt_id,
               {_time_key('"Stop:Arrival time"')}       AS arrival_time,
               {_time_key('"Stop:Departure time"')}     AS departure_time,
               station_code,
               "Service:Train number"                   AS train_id,
               date_diff('day', "Service:Date", "Stop:Arrival time"::DATE)::SMALLINT   AS
 ↪  arrival_day_offset,
               date_diff('day', "Service:Date", "Stop:Departure time"::DATE)::SMALLINT AS
 ↪  departure_day_offset,
               CASE WHEN coalesce("Stop:Arrival cancelled", true)
                     AND coalesce("Stop:Departure cancelled", true) THEN 'completely'
                    WHEN "Stop:Arrival cancelled"                   THEN 'arrival-only'
                    WHEN "Stop:Departure cancelled"                 THEN 'departure-only'
                    ELSE 'none' END                     AS cancel_status,
               CASE WHEN NOT "Stop:Arrival cancelled"   THEN "Stop:Arrival delay"
 ↪  END::INTEGER AS arrival_delay,
               CASE WHEN NOT "Stop:Departure cancelled" THEN "Stop:Departure delay"
 ↪  END::INTEGER AS departure_delay
        FROM stops
    """,
}


def _validate(con, out: Path) -> dict:
    """Validate transformed tables against data integrity and foreign key constraints.

    Args:
        con: Active DuckDB database connection containing staged views.
        out: Path to the directory containing transformed Parquet files.

    Returns:
        dict: Table row counts if all validation checks succeed.

    Raises:
        ValueError: If any integrity constraint or foreign key check fails.
    """
    for t in config.TABLES:
        con.execute(f"CREATE OR REPLACE VIEW {t} AS FROM read_parquet({_sql_str(out /
 ↪  f'{t}.parquet')})")
    counts = {t: con.execute(f"SELECT count(*) FROM {t}").fetchone()[0] for t in config.TABLES}

    checks = {
        "fact_stop rows != staged rows": f"SELECT {counts['fact_stop']} <> (SELECT count(*)
         ↪  FROM stops)",
        "dim_station: code with several names": """
            SELECT count(*) > 0 FROM (SELECT station_code FROM stops GROUP BY 1 HAVING
 ↪  count(DISTINCT station_name) > 1)
        """,
        "fact_service: date_key not in dim_date": """
            SELECT count(*) > 0 FROM fact_service s ANTI JOIN dim_date d USING (date_key)
        """,
        "fact_stop: rdt_id not in fact_service": """
            SELECT count(*) > 0 FROM fact_stop f ANTI JOIN fact_service s USING (rdt_id)
        """,
```

```
        "fact_stop: station_code not in dim_station": """
            SELECT count(*) > 0 FROM fact_stop f ANTI JOIN dim_station s USING (station_code)
        """,
        "fact_stop: arrival/departure_time not in dim_time": """
            SELECT count(*) > 0 FROM fact_stop
            WHERE (arrival_time IS NOT NULL AND arrival_time NOT IN (SELECT time_key FROM
↪   dim_time))
               OR (departure_time IS NOT NULL AND departure_time NOT IN (SELECT time_key FROM
↪   dim_time))
        """,
        "dim_time: hour without time_of_day": "SELECT count(*) > 0 FROM dim_time WHERE
         ↪  time_of_day IS NULL",
    }
    problems = [name for name, sql in checks.items() if con.execute(sql).fetchone()[0]]
    if problems:
        raise ValueError(f"Transform failed checks: {'; '.join(problems)}. Counts: {counts}")
    return counts


def run_transform(staged: Path, out: Path) -> dict:
    """Build every table from the staged Parquet into `out/<table>.parquet`.

    Each file is written to a `.part` first and renamed, so a failed run never leaves a
    half-written table next to good ones.

    Args:
        staged: Staged services Parquet from the extract.
        out: Warehouse directory for this year (config.warehouse_dir).

    Returns:
        dict: The output directory and the row count of each table.
    """
    out.mkdir(parents=True, exist_ok=True)
    con = _connect()
    try:
        con.execute(
            f"""
            CREATE VIEW stops AS
            SELECT *,
                   {_station_code_sql()} AS station_code,
                   "Stop:Station name" AS station_name,
                   (coalesce("Stop:Arrival time", "Stop:Departure time"), "Stop:RDT-ID") AS
↪   stop_order
            FROM read_parquet({_sql_str(staged)})
            """
        )
        for table in config.TABLES:
            dest = out / f"{table}.parquet"
            part = dest.with_name(dest.name + ".part")
            con.execute(f"COPY ({_QUERIES[table]}) TO {_sql_str(part)} (FORMAT parquet,
↪   COMPRESSION zstd)")
            os.replace(part, dest)
            log.info("Wrote %s (%.1f MB)", dest, dest.stat().st_size / 1e6)
        counts = _validate(con, out)
    finally:
        con.close()
```

```
    log.info("Transform counts: %s", counts)
    return {"warehouse_dir": str(out), **counts}
```

etl/load.py

```
"""Load stage for the Dutch Railway ETL pipeline.

Loads transformed dimensional and fact tables into PostgreSQL (dutch_railway_db)
using streaming COPY commands within an atomic transaction.
"""

import logging
import tempfile
from pathlib import Path

import psycopg2

import etl_config as config
from etl.extract import _connect, _sql_str

log = logging.getLogger(__name__)

SCHEMA_SQL = (
    Path(__file__).parent / "sql" / "schema.sql"
    if (Path(__file__).parent / "sql" / "schema.sql").exists()
    else Path(__file__).with_name("schema.sql")
)

_DIM_KEYS = {"dim_date": "date_key", "dim_time": "time_key", "dim_station": "station_code"}


def _columns(parquet: Path) -> list[str]:
    """Inspect and return column names from a Parquet file using DuckDB.

    Args:
        parquet: Path to the target Parquet file.

    Returns:
        list[str]: Column names in schema order.
    """
    con = _connect()
    try:
        return [r[0] for r in con.execute(f"DESCRIBE SELECT * FROM
        ↪  read_parquet({_sql_str(parquet)})").fetchall()]
    finally:
        con.close()


def _to_csv(parquet: Path, csv: Path) -> None:
    """Export Parquet table data to headerless CSV format for PostgreSQL COPY.

    Args:
        parquet: Source Parquet file path.
```

```
        csv: Destination CSV file path.
    """
    con = _connect()
    try:
        con.execute(f"COPY (FROM read_parquet({_sql_str(parquet)})) TO {_sql_str(csv)} (FORMAT
↪   csv, HEADER false)")
    finally:
        con.close()


def _copy(cur, table: str, columns: list[str], csv: Path) -> None:
    """Stream CSV rows into PostgreSQL table via COPY FROM STDIN.

    Args:
        cur: Active psycopg2 database cursor.
        table: Destination database table name.
        columns: Ordered list of column names matching CSV fields.
        csv: Path to the headerless CSV file to stream.
    """
    with open(csv, encoding="utf-8") as f:
        cur.copy_expert(f"COPY {table} ({', '.join(columns)}) FROM STDIN WITH (FORMAT csv)", f)


_FACTS = ("fact_service", "fact_stop")


def _fact_foreign_keys(cur) -> list[tuple[str, str, str]]:
    """Query foreign key constraints defined on fact tables from the catalog.

    Args:
        cur: Active psycopg2 database cursor.

    Returns:
        list[tuple[str, str, str]]: List of (table_name, constraint_name, definition).
    """
    cur.execute(
        """
        SELECT conrelid::regclass::text, conname, pg_get_constraintdef(oid)
        FROM pg_constraint
        WHERE contype = 'f' AND conrelid::regclass::text = ANY(%s)
        ORDER BY 1, 2
        """,
        (list(_FACTS),),
    )
    return cur.fetchall()


def _fact_secondary_indexes(cur) -> list[tuple[str, str]]:
    """Query non-constraint secondary indexes defined on fact tables.

    Args:
        cur: Active psycopg2 database cursor.

    Returns:
        list[tuple[str, str]]: List of (index_name, create_index_sql).
    """
```

```
    cur.execute(
        """
        SELECT indexname, indexdef FROM pg_indexes
        WHERE schemaname = current_schema() AND tablename = ANY(%s)
          AND indexname NOT IN (SELECT conname FROM pg_constraint)
        ORDER BY 1
        """,
        (list(_FACTS),),
    )
    return cur.fetchall()


def load_warehouse(warehouse: Path, year: int, db_url: str = config.DB_URL) -> dict:
    """Load every table in `warehouse` into Postgres, replacing the facts of `year`.

    Args:
        warehouse: Directory with the transform's `<table>.parquet` files.
        year: Year of the facts in `warehouse`; its existing facts are replaced.
        db_url: libpq connection URL. Defaults to config.DB_URL.

    Returns:
        dict: Row counts per table in the database after the load.
    """
    year_range = (year * 10000 + 101, year * 10000 + 1231)

    with tempfile.TemporaryDirectory(dir=warehouse, prefix=".load-") as tmp:
        csvs = {}
        for table in config.TABLES:
            parquet = warehouse / f"{table}.parquet"
            csvs[table] = (Path(tmp) / f"{table}.csv", _columns(parquet))
            _to_csv(parquet, csvs[table][0])

        conn = psycopg2.connect(db_url)
        try:
            with conn, conn.cursor() as cur:
                cur.execute(SCHEMA_SQL.read_text())

                for table, key in _DIM_KEYS.items():
                    csv, cols = csvs[table]
                    cur.execute(f"CREATE TEMP TABLE {table}_in (LIKE {table}) ON COMMIT DROP")
                    _copy(cur, f"{table}_in", cols, csv)
                    updates = ", ".join(f"{c} = EXCLUDED.{c}" for c in cols if c != key)
                    cur.execute(
                        f"INSERT INTO {table} ({', '.join(cols)}) SELECT {', '.join(cols)} FROM
                         ↪  {table}_in "
                        f"ON CONFLICT ({key}) DO UPDATE SET {updates}"
                    )
                    log.info("Upserted %s: %d rows", table, cur.rowcount)

                fks = _fact_foreign_keys(cur)
                for table, name, _ in fks:
                    cur.execute(f"ALTER TABLE {table} DROP CONSTRAINT {name}")

                cur.execute(
                    "DELETE FROM fact_stop WHERE rdt_id IN "
                    "(SELECT rdt_id FROM fact_service WHERE date_key BETWEEN %s AND %s)",
```

```
                    year_range,
                )
                log.info("Deleted %d existing fact_stop rows for %d", cur.rowcount, year)
                cur.execute("DELETE FROM fact_service WHERE date_key BETWEEN %s AND %s",
↪   year_range)
                log.info("Deleted %d existing fact_service rows for %d", cur.rowcount, year)

                indexes = _fact_secondary_indexes(cur)
                for name, _ in indexes:
                    cur.execute(f"DROP INDEX {name}")

                for table in ("fact_service", "fact_stop"):
                    csv, cols = csvs[table]
                    _copy(cur, table, cols, csv)
                    log.info("Copied %s: %d rows", table, cur.rowcount)

                for _, definition in indexes:
                    cur.execute(definition)
                for table, name, definition in fks:
                    cur.execute(f"ALTER TABLE {table} ADD CONSTRAINT {name} {definition}")
                log.info("Rebuilt %d indexes and %d foreign keys", len(indexes), len(fks))

                counts = {}
                for table in config.TABLES:
                    cur.execute(f"SELECT count(*) FROM {table}")
                    counts[table] = cur.fetchone()[0]
            conn.autocommit = True
            with conn.cursor() as cur:
                cur.execute("ANALYZE")
        finally:
            conn.close()

    log.info("Database counts: %s", counts)
    return counts
```

etl/dump.py

```
"""Dump stage for the Dutch Railway ETL pipeline.

Snapshots the warehouse tables in PostgreSQL (dutch_railway_db) into a pg_dump
custom-format archive, so others can restore the database with
scripts/setup_db.py instead of re-running the whole pipeline.
"""

import logging
import subprocess
from pathlib import Path

import etl_config as config

log = logging.getLogger(__name__)


def dump_database(dump: Path = config.DUMP_PATH, db_url: str = config.DB_URL) -> dict:
```

```
    """Write the warehouse tables to `dump`, replacing any previous dump atomically.

    Args:
        dump: Destination `.dump` file (pg_dump custom format).
        db_url: libpq connection URL. Defaults to config.DB_URL.

    Returns:
        dict: The dump path and its size in bytes.
    """
    dump.parent.mkdir(parents=True, exist_ok=True)
    tmp = dump.with_name(f".{dump.name}.tmp")
    cmd = [
        "pg_dump",
        "--format=custom",
        "--no-owner",
        "--no-privileges",
        f"--file={tmp}",
        *(f"--table={table}" for table in config.TABLES),
        f"--dbname={db_url}",
    ]
    try:
        subprocess.run(cmd, check=True, capture_output=True, text=True)
    except subprocess.CalledProcessError as e:
        tmp.unlink(missing_ok=True)
        raise RuntimeError(f"pg_dump failed: {e.stderr.strip()}") from e
    tmp.replace(dump)

    size = dump.stat().st_size
    log.info("Dumped %s to %s (%.1f MB)", ", ".join(config.TABLES), dump, size / 1e6)
    return {"dump": str(dump), "bytes": size}
```